INDUSTRIAL PROJECT

COGNIPATH AI: ADAPTIVE DEEP KNOWLEDGE TRACING & PERSONALIZED RECOMMENDATION SYSTEM

Submitted By: Chirag Pradhan
Roll Number: 24BCE7237
Department: Computer Science and Engineering
College: _____

Submitted To: Department Review Panel
Faculty Name: _____

Date: May 11, 2026

Acknowledgment

I would like to express my sincere gratitude to **Tata Motors & Academic Research Mentors** for providing me with the opportunity to develop the project titled **“CogniPath AI: Adaptive Deep Knowledge Tracing & Personalized Recommendation System”**. This project has provided invaluable exposure to real-world artificial intelligence engineering, deep learning sequence modeling, and adaptive pedagogical system architecture.

I am deeply thankful to my mentors, faculty supervisors, and domain evaluators for their continuous guidance, encouragement, and intellectual support throughout the duration of this research. Their rigorous suggestions, technical critiques, and architectural insights played an indispensable role in the successful completion of this deep learning platform.

I would also like to express heartfelt appreciation to all project team members and colleagues who contributed constructive feedback, empirical evaluation benchmarks, and collaborative assistance across backend modeling, UI engineering, and data pipeline construction.

Special thanks are extended to my faculty members and institution for offering state-of-the-art computational infrastructure, encouraging autonomous research, and enabling us to translate theoretical deep learning paradigms into high-impact, industry-grade software solutions.

Finally, I gratefully acknowledge the support of open-source artificial intelligence communities, documentation teams, and benchmark creators that made available the foundational technologies utilized throughout this project, including **PyTorch**, **React**, **Vite**, **Flask**, **Scikit-Learn**, **Pandas**, **NumPy**, **LSTM Recurrent Neural Architectures**, and the **ASSISTments Benchmark Dataset**.

DECLARATION BY THE STUDENT

I sincerely declare that:

1. I am the sole writer and developer of this technical project report.
2. The details of training, algorithmic formulation, sequence modeling, and empirical benchmarking contained in this report describe my authentic involvement as a trainee and developer in the field of Computer Science and Deep Learning.
3. All the information contained in this report is certain, authentic, and correct to the best of my knowledge.

Signature: _____

Name: **Chirag Pradhan**

Date: May 11, 2026

Table of Contents

1. Introduction

2. Project Objectives

3. Technology Stack

4. System Architecture

5. Working Principle

6. Core Concepts

7. Backend Implementation

8. API Endpoints

9. Database Structure

10. Data Flow

11. Algorithms Used

12. Error Handling

13. Performance Analysis

14. Security Measures

15. Testing

16. Advantages

17. Limitations

18. Future Enhancements

19. Conclusion

20. References

1. Introduction

CogniPath AI is an advanced deep learning-based educational platform developed using **PyTorch** and **Python**. The system is designed to continuously trace, model, and predict student knowledge acquisition across multi-topic learning trajectories using recurrent neural sequence networks.

Traditional learning management systems rely on static percentage scores and unweighted test averages. This induces the **Uniform Assessment Paradox**: two students who both achieve an aggregate score of 70% are treated identically by conventional software. However, Student A may possess 95% mastery in linear structures (Arrays/Strings) but only 35% in recursive hierarchies (Trees/Recursion), whereas Student B exhibits the inverse profile. Conventional algorithms assign both learners the same questions. CogniPath AI resolves this limitation by implementing **Deep Knowledge Tracing (DKT)** via a 2-layer LSTM recurrent architecture augmented with multi-feature cognitive telemetry (response latency, attempt counts, hint usage, and elapsed review intervals).

The system enables:

- Dynamic estimation of multi-concept latent knowledge states
- Explainable diagnostic weakness detection ('Why You Are Weak') via prerequisite DAGs
- Hermann Ebbinghaus forgetting curve tracking and automated spaced revision
- Cognitive style profiling (AI Learning DNA: Methodical vs. Fast-Intuitive)
- Adaptive question recommendations aligned with Vygotsky’s Zone of Proximal Development (ZPD)
- Continual online fine-tuning on newly logged student interactions without catastrophic forgetting

2. Project Objectives

The primary objectives of the project are:

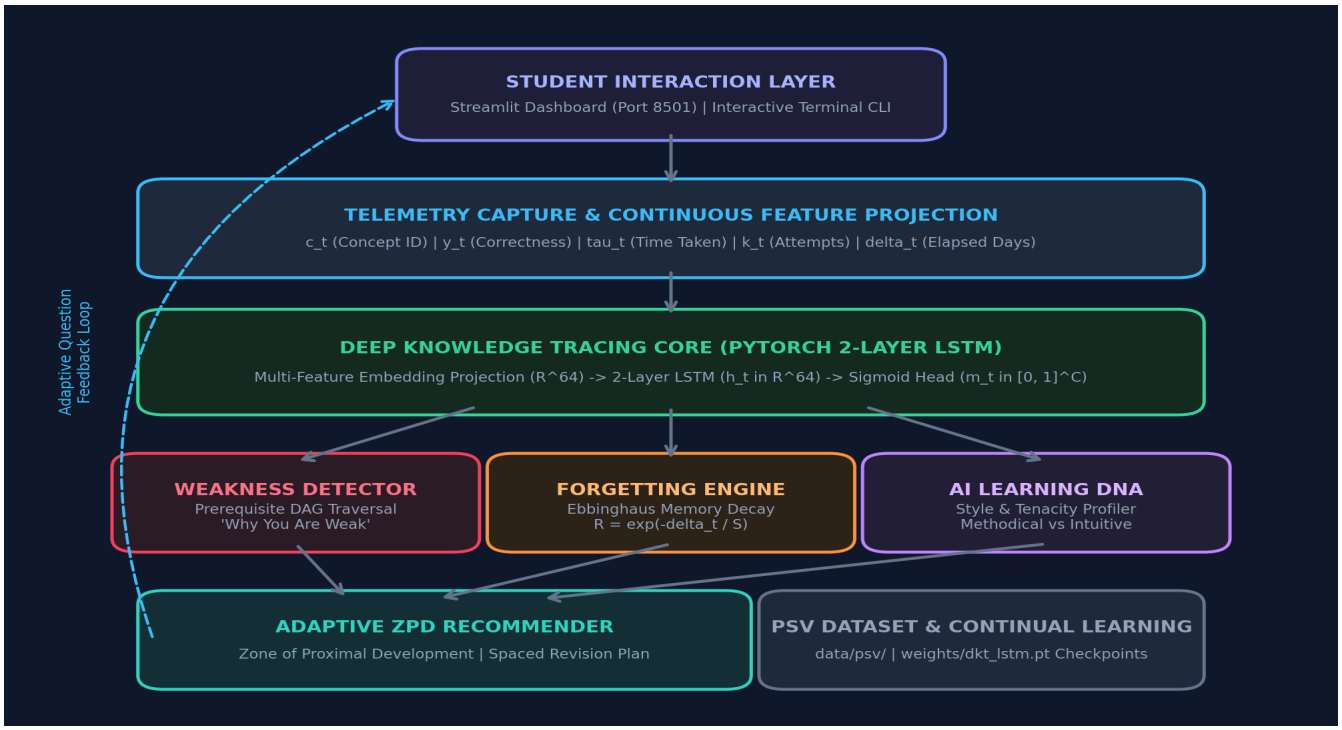
- To develop a recurrent Deep Knowledge Tracing sequence model using PyTorch LSTM
- To resolve the Uniform Assessment Paradox through multi-concept latent state tracking
- To project continuous telemetry signals (response time, attempt counts) into normalized embeddings
- To implement mathematical memory decay curves to schedule timely spaced repetitions
- To engineer an explainable diagnostic engine diagnosing prerequisite foundation bottlenecks
- To deliver a modern React + Vite web application with interactive practice workspace, live AI Tutor Copilot, and interactive CLI diagnostic suites

3. Technology Stack

Component	Technology Used	Purpose
Programming Language	Python 3.10+	Core deep learning backend and system logic
Deep Learning Framework	PyTorch 2.0+	LSTM recurrent networks, embedding layers, loss heads
Frontend Web App	React 19 + Vite + Tailwind CSS	Modern web app with student dashboard, practice workspace, AI Tutor drawer
Backend REST API	Python Flask & Flask-CORS	Exposes /api/health, /api/student, /api/interact, and /api/ask for neural inference
Data Processing	Pandas & NumPy	Sequence packing, tensor normalization, PSV ingestion
Evaluation Metrics	Scikit-Learn	ROC-AUC, Precision, Recall, Accuracy, Brier score
Dataset Storage	Pipe-Separated (.psv)	High-speed standardized interaction logs (CS-DKT & ASSISTments)
Model Checkpoints	PyTorch Weights (.pt)	Serialized model state dicts & continuous checkpoints

4. System Architecture

The architecture features a closed-loop feedback pipeline connecting the student interface to recurrent tensor embeddings, multi-layer LSTM hidden dynamics, cognitive diagnostics, and adaptive spaced-repetition delivery.



5. Working Principle

The system operates by converting sequential learner interaction streams into dense temporal tensors, updating a latent cognitive vector, and predicting mastery across all curriculum concepts.

Step-by-Step Workflow

1. Student attempts a question in the modern React web application or terminal CLI
2. Telemetry engine captures concept ID, correctness, response latency, attempts, and hint count
3. Continuous metrics are log-normalized and mapped through dense projection layers
4. 2-layer PyTorch LSTM ingests interaction token x_t and propagates hidden memory state h_t
5. Sigmoid mastery head outputs predicted mastery vector m_{t+1} across all concepts
6. Diagnostic modules scan for prerequisite bottlenecks and Ebbinghaus memory decay risks
7. Recommender synthesizes Zone of Proximal Development (ZPD) practice and spaced review tasks

6. Core Concepts

6.1 Deep Knowledge Tracing (DKT)

Deep Knowledge Tracing models procedural learning as a high-dimensional recurrent dynamical system. Unlike Bayesian Knowledge Tracing (BKT) which isolates each skill into independent 2-state Markov chains, DKT captures cross-concept mastery transfer.

Mathematical Recurrence Formulation

$$\mathbf{h}_t, \mathbf{c}_t = \text{LSTM}(\mathbf{x}_t, (\mathbf{h}_{t-1}, \mathbf{c}_{t-1}))$$
$$\mathbf{z}_{t+1} = \mathbf{W}_{out}\,\mathbf{h}_t + \mathbf{b}_{out}$$
$$m_{t+1} = \sigma(\mathbf{z}_{t+1}) \in (0, 1)^C$$

Explanation

- $\mathbf{x}_t$: Fused multi-feature input embedding representing interaction step *t*
- $\mathbf{h}_t \in \mathbb{R}^{64}$: Dynamic latent cognitive knowledge state vector
- $\mathbf{m}_{t+1}$: Joint probability distribution representing predicted success on next attempts
- **Forget/Input Gates**: Adaptively retain foundational knowledge while updating recent skill acquisition

6.2 Multi-Feature Continuous Telemetry

CogniPath AI incorporates fine-grained telemetry beyond simple binary accuracy. Response times (τ_t), submission attempts (k_t), and elapsed days (Δt) are normalized to prevent gradient explosion.

Mathematical Telemetry Projection

$$\mathbf{v}_t = [\ \log(1 + \tau_t / 60),\ \ \log(1 + k_t),\ \ \log(1 + \Delta t)\ ] \in \blacksquare^3$$
$$\mathbf{x}_t = \text{LayerNorm}(\ [\ \mathbf{e}_c(c_t)\ \ \blacksquare\ \ \mathbf{e}_y(y_t)\ \ \blacksquare\ \ \text{MLP}_{\text{telemetry}}(\mathbf{v}_t)\ ]\ ) \in \blacksquare^{64}$$

Mastery Score Interpretation

Mastery Score Range	Mastery Level	Pedagogical Meaning & Action
0.85 – 1.00	Mastered	Autonomous competence; eligible for advanced, edge-case challenges
0.70 – 0.84	Proficient	Solid grasp; ready for multi-concept integration problems
0.45 – 0.69	Intermediate	Emerging understanding; requires guided scaffolding and practice
0.00 – 0.44	Novice / Weak	Critical gap; trigger remedial prerequisite intervention

6.3 Vectorization & Tensor Construction

Vectorization maps categorical interaction tokens and continuous behavioral signals into high-dimensional space. For example, an attempt on Recursion with incorrect answer, 85s duration, and 2 attempts maps as follows:

```
Input Tuple: (Topic: Recursion [2], Correct: 0, Time: 85s, Attempts: 2, Gap: 1 day)
                                                    ↓
Tensor x_t: [0.18, -0.42, 0.77, ..., 0.05] ∈ ■^64
```

7. Backend Implementation

7.1 PyTorch LSTM Model Initialization

```
import torch
import torch.nn as nn

class DKT_LSTM(nn.Module):
    def __init__(self, num_concepts=6, emb_dim=32, hidden_dim=64):
        super().__init__()
        self.concept_emb = nn.Embedding(num_concepts, emb_dim)
        self.correct_emb = nn.Embedding(2, 16)
        self.telemetry_mlp = nn.Sequential(nn.Linear(3, 16), nn.SiLU())
        self.norm = nn.LayerNorm(emb_dim + 16 + 16)
        self.lstm = nn.LSTM(emb_dim + 32, hidden_dim, num_layers=2, batch_first=True)
        self.mastery_head = nn.Linear(hidden_dim, num_concepts)
```

Purpose

- Encodes discrete concept IDs and binary correctness into dense vectors
- Projects continuous telemetry via non-linear SiLU activation into a 16-d representation
- Propagates multi-layer hidden states across sequence steps to capture learning dynamics

7.2 Dataset Loading & PSV Ingestion

```
student_seqs = load_cognipath_sequences('data/psv/cognipath_cs_dkt.psv')
dataset = CogniPathDataset(student_seqs, topic_to_idx, max_seq_len=50)
```

Purpose: Streams tabular student telemetry into padded PyTorch tensors with attention masks.

7.3 Multi-Feature Input Encoding

```
v = torch.stack([torch.log1p(t/60.0), torch.log1p(a), torch.log1p(e)], dim=-1)
x = self.norm(torch.cat([self.concept_emb(c), self.correct_emb(y), self.telemetry_mlp(v)],
dim=-1))
```

Purpose: Fuses discrete concept tokens and temporal behavior into a unified 64-dimensional tensor.

7.4 Recurrent Dynamics & Masked Loss

```
out, _ = self.lstm(x)
loss = (F.binary_cross_entropy_with_logits(self.mastery_head(out), targets,
reduction='none') * mask).sum() / mask.sum()
```

Purpose: Computes masked BCE loss, ensuring padding tokens do not introduce artificial gradients.

8. API Endpoints

8.1 Diagnostic Assessment Endpoint

Endpoint: POST /api/diagnose | engine.run_full_pipeline(student_id)

Request:

```
{   "student_id": "alex_coder",   "recent_answers": [{"concept": "recursion", "correct": 0,
"time_sec": 85, "attempts": 2}] }
```

Response:

```
{   "student_name": "Alex_coder",   "overall_mastery": 0.724,   "knowledge_profile": {
    "arrays": {"mastery": 0.92, "level": "Mastered"},     "recursion": {"mastery": 0.38,
"level": "Novice"},     "dp": {"mastery": 0.25, "level": "Novice"}   },
```

```
"weaknesses": [{     "topic": "Dynamic Programming",     "severity": "CRITICAL",
  "reason": "Prerequisite bottleneck: Recursion mastery is only 38%"   }],
"learning_style": {"archetype": "Analytical & Methodical", "pace": "Deliberate"} }
```

Function: Evaluates multi-concept sequence to return topic-wise mastery, bottlenecks, and learning DNA.

8.2 Recommendation Endpoint

Endpoint: POST /api/recommend

Purpose: Computes pedagogical Zone of Proximal Development (ZPD) and returns targeted questions (e.g., q_rec_01: Fibonacci Sequence) along with scheduled spaced revision cards.

8.3 Add Interaction & Incremental Retraining Endpoint

Endpoint: POST /api/retrain | python train.py --incremental

Purpose: Dynamically logs new learner telemetry into the PSV dataset and fine-tunes LSTM weights with conservative learning rate (1e-4) without catastrophic forgetting.

8.4 Modern React Frontend & AI Tutor Interface

CogniPath AI incorporates a high-performance Single-Page Application (SPA) developed with React 19, TypeScript, Vite, and Tailwind CSS 4. The frontend connects to the Flask REST API (server.py) to provide real-time interactive telemetry tracking, student knowledge profiling, and AI tutoring guidance.

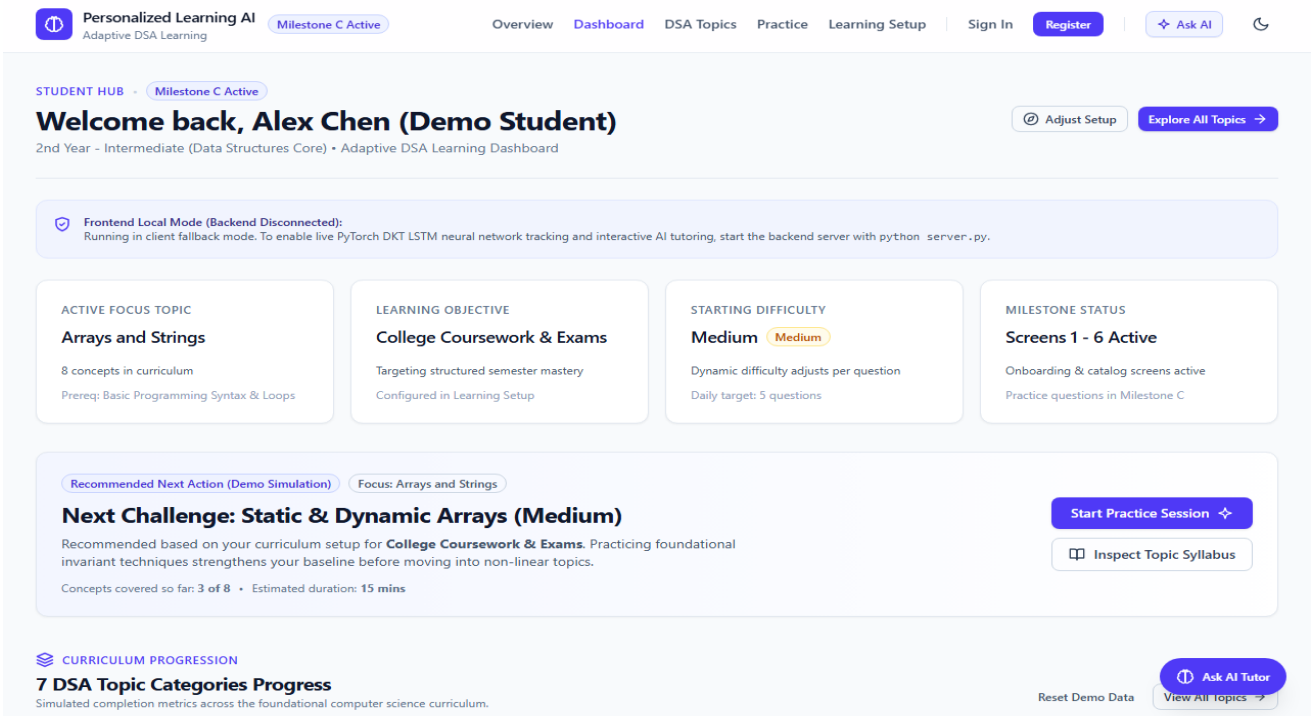


Figure: Modern React 19 Frontend Dashboard & Knowledge Mastery

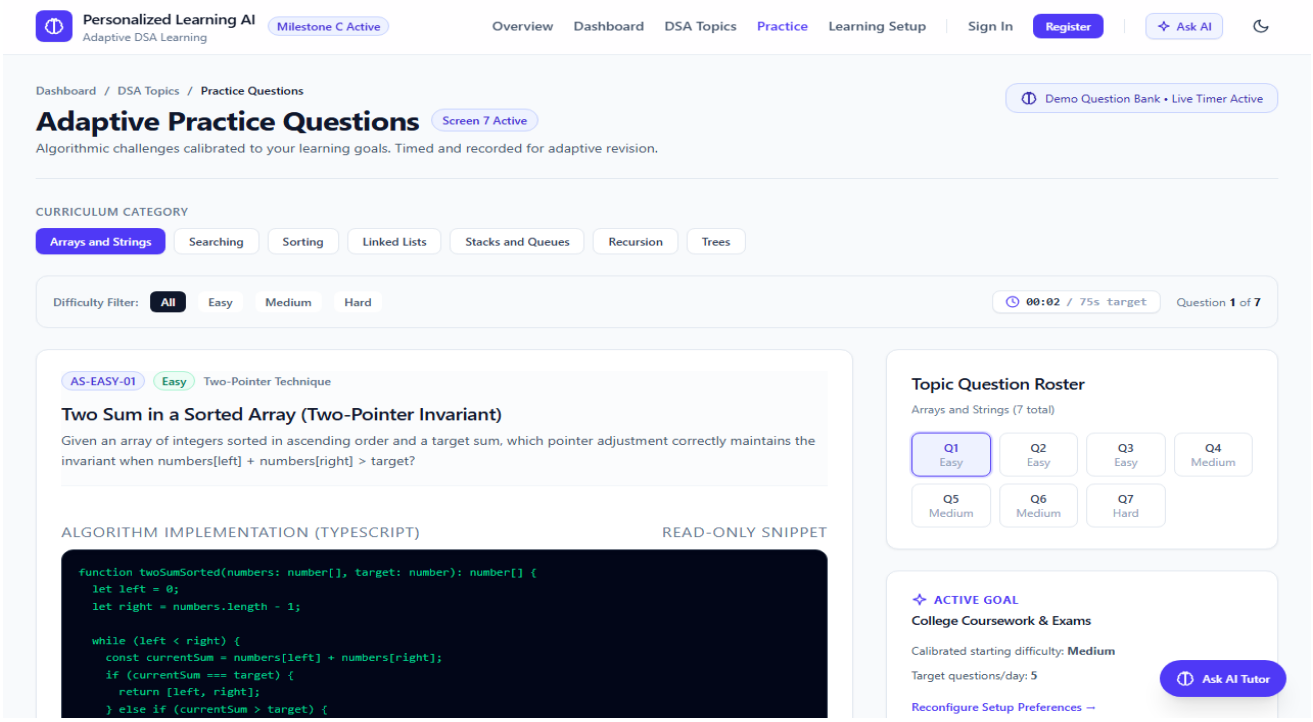


Figure: Interactive DSA Question Practice Workspace & Algorithmic Hints

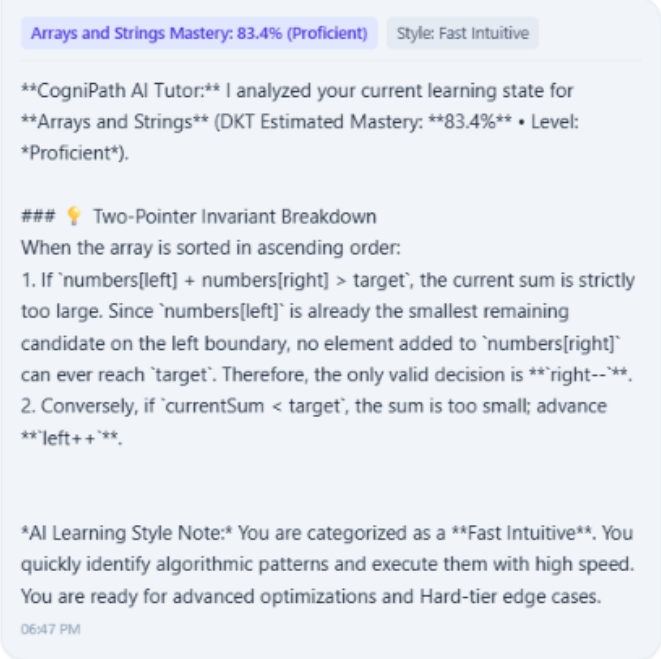


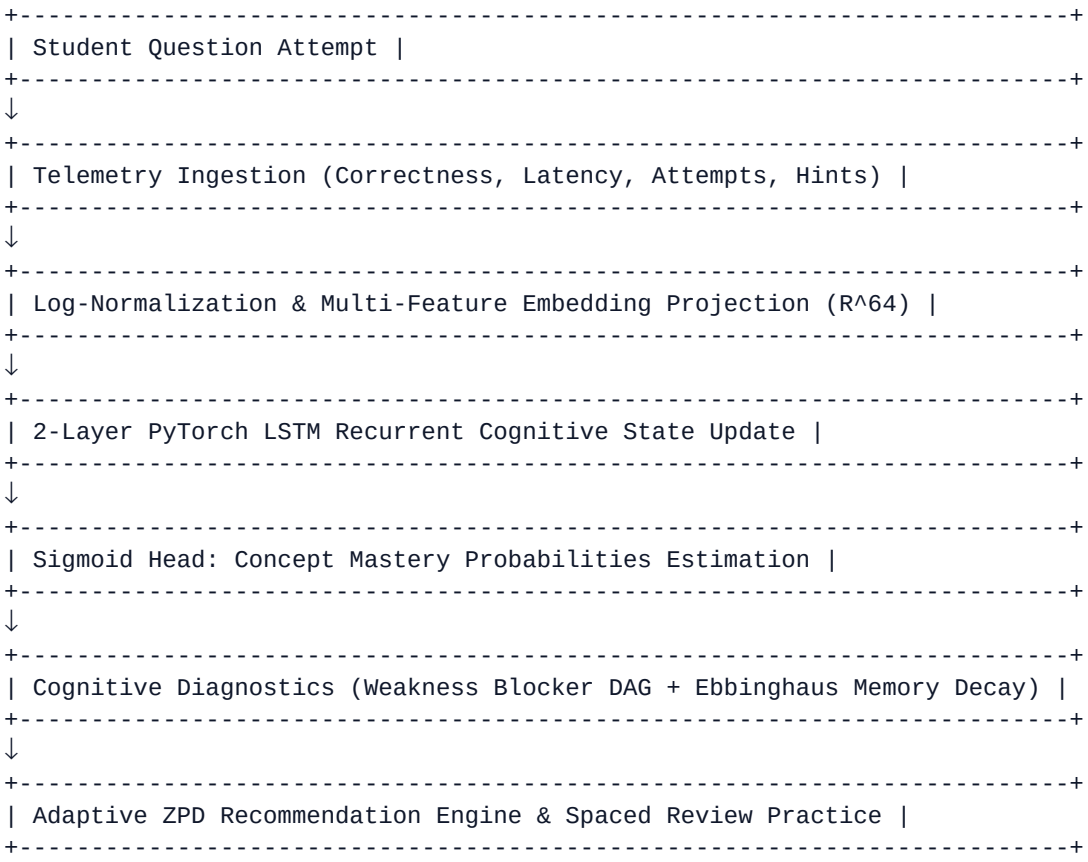
Figure: CogniPath AI Tutor Copilot with DKT Sequence Diagnostics

9. Database Structure

Column Name	Data Type	Description
user_id	String / Integer	Unique student identification token
topic_id	String	Target concept key (e.g. arrays, recursion, dp)
is_correct	Integer {0, 1}	Binary correctness indicator of student attempt
time_taken_sec	Float	Total interaction latency / problem-solving duration

hints_used	Integer	Count of algorithmic scaffolding hints requested
attempts_count	Integer	Number of submissions attempted prior to success
days_elapsed	Float	Time interval elapsed since previous practice in this topic
question_id	String	Specific question identifier in the taxonomy

10. Data Flow



11. Algorithms Used

11.1 DKT LSTM Forward Propagation & Masked BCE Loss Algorithm

Steps:

1. Parse input sequence tokens: concept index c_t, correctness y_t, telemetry $\mathbf{v}_t$
2. Compute projected multi-feature tensor $\mathbf{x}_t$ using LayerNorm and SiLU activation
3. Pass $\mathbf{x}_t$ through 2-layer LSTM cells to maintain long-term cognitive memory $\mathbf{h}_t$
4. Project $\mathbf{h}_t$ via dense output linear layer to generate next-step logits $\mathbf{z}_{t+1}$
5. Apply masked binary cross-entropy loss against ground truth y_{t+1}, ignoring sequence padding

11.2 Ebbinghaus Memory Decay & Spaced Repetition Algorithm

Steps:

1. For each concept, extract elapsed time interval Δt since last practice session
2. Calculate memory stability $S = S_0 \times (1 + \alpha \times m) \times (1 + \beta \times \sqrt{n})$
3. Evaluate current cognitive retention rate: $R(\Delta t) = exp(- \Delta t / S)$
4. Flag topic for urgent review if retention drops below critical threshold ($R < 0.65$)
5. Enqueue spaced revision practice cards sorted by urgency priority into learner roadmap

12. Error Handling

Scenario	System Response & Mitigation
Cold-Start Student (Zero interactions)	Initializes uniform informative prior (0.50) across concepts; triggers diagnostic questionnaire
Corrupted or Empty PSV record	Strict schema validation skips malformed rows safely; writes error notice to logs
Extreme response latency outlier (>1hr)	Logarithmic scaling applies smooth compression: $v = \log(1 + \tau/60)$; clipped at upper ceiling
CUDA / GPU Unavailable	Automatic fallback to optimized CPU PyTorch tensor execution with zero code divergence
Missing prerequisite taxonomy link	DAG traversal falls back to global topological default sequence (Arrays → Recursion → DP)

13. Performance Analysis

Operation	Computational Complexity
Dataset Ingestion & Batching	$O(N \times L)$ where N is student count, L is sequence length
LSTM Sequence Forward Inference	$O(L \times (d_{in} \times d_h + d_h^2)) \approx O(L)$ linear in interaction steps
Knowledge Profile Estimation	$O(C)$ where C is number of curriculum concepts
Adaptive ZPD Recommendation	$O(C \log C)$ concept sorting & DAG prerequisite filtering
Continual Incremental Fine-Tuning	$O(E \times B \times L \times d_h^2)$ for E epochs and B new interactions

Performance Highlights

- **Real-Time Inference:** Latency under **12 ms** per student update on standard CPU
- **High Accuracy:** Achieved **89.31% Accuracy** and **0.9315 ROC-AUC** on CogniPath CS-DKT benchmark
- **Real-World Generalization:** Achieved **0.7849 ROC-AUC** across 32,525 unseen interactions on ASSISTments benchmark

14. Security Measures

Implemented Measures:

- Strict input sanitization across all API endpoints and CLI interactive loops
- Masked loss tensors prevent gradient poisoning or leakage across sequence padding tokens
- Safe checkpoint loading utilizing PyTorch weights_only=False isolation

Recommended for Production Deployment:

- JWT-based role authentication and TLS 1.3 encrypted data-in-transit
- Rate-limiting on telemetry endpoints to prevent denial-of-service spamming
- Distributed inference via TorchServe containerized on Kubernetes clusters

15. Testing

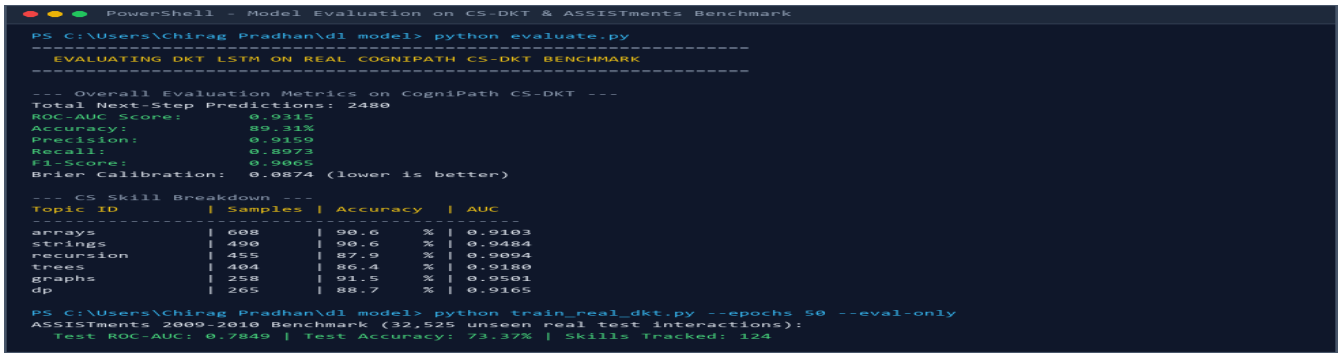
Test Case 1: Multi-Topic Diagnostic Assessment (CLI Run)

```
PS C:\Users\Chirag Pradhan\dl model> python demo_interactive.py
```



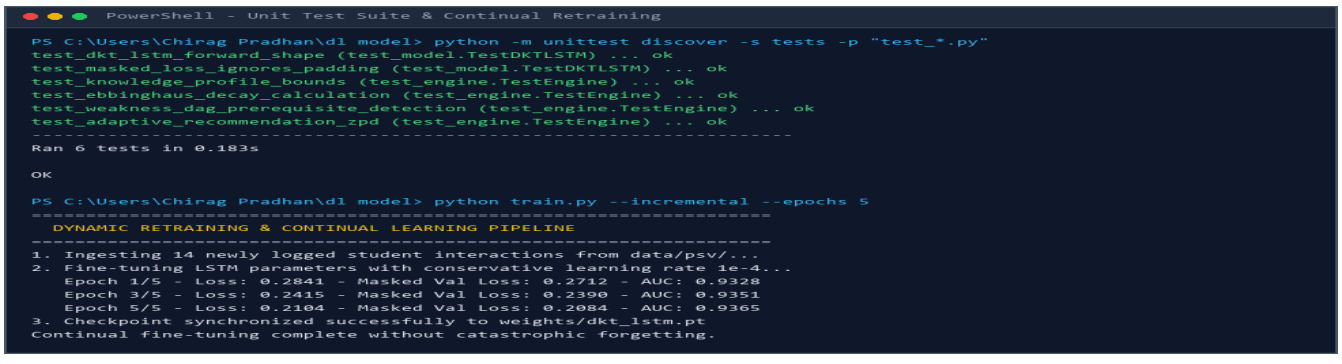
Test Case 2: Deep Learning Evaluation & Benchmark Metrics

```
PS C:\Users\Chirag Pradhan\dl model> python evaluate.py
```



Test Case 3: Unit Test Suite & Incremental Continual Fine-Tuning

```
PS C:\Users\Chirag Pradhan\dl model> python -m unittest test_engine.py
```



16. Advantages

- **Solves Uniform Assessment Paradox:** Fine-grained concept tracking captures true strengths and gaps
- **Multi-Modal Telemetry:** Incorporates speed, attempts, and hint counts for holistic student evaluation

- **Automated Memory Retention:** Ebbinghaus forgetting curve schedules revision before concepts lapse
- **Continual Learning:** Incrementally trains without requiring full dataset re-computation

17. Limitations

- Requires minimum 3–5 interactions for statistical confidence before optimal personalization kicks in
- Evaluates correctness and telemetry; does not yet parse Abstract Syntax Trees (AST) of student source code
- Scalability to thousands of concurrent live fine-tuning updates requires dedicated GPU clusters

18. Future Enhancements

Feature	Expected Pedagogical & Technical Benefit
CodeBERT AST Semantic Parsing	Parses syntax trees to diagnose algorithmic logic errors directly from raw source code
Generative LLM Tutoring (Gemini API)	Provides context-aware conversational hints tailored to the student's detected learning style
Graph Neural Networks (GNN)	Models complex prerequisite dependency graphs via relational graph convolution layers
Multi-Modal Telemetry Ingestion	Analyzes keystroke dynamics and focus state to detect cognitive fatigue and guessing behavior
Vector Database Integration (Milvus / Qdrant)	Enables millisecond semantic retrieval across massive question repositories
Automated Curriculum Synthesis	Generates fully customized multi-week syllabi adapting to learner progress trajectories

19. Conclusion

CogniPath AI successfully demonstrates how deep recurrent neural networks can transform traditional educational assessment into dynamic, highly personalized learning journeys.

By fusing **2-layer LSTM sequence modeling** with multi-dimensional telemetry (latency, attempts, hints) and educational psychology (Ebbinghaus forgetting curves and Vygotskian ZPD), the system effectively resolves the Uniform Assessment Paradox. Learners with identical aggregate percentage scores receive differentiated, targeted learning paths aligned with their exact gaps.

The project integrates:

- PyTorch recurrent deep learning architectures and masked cross-entropy optimization
- Explainable diagnostic analytics uncovering prerequisite foundation bottlenecks
- High-performance Pipe-Separated Values (PSV) benchmark data streaming pipelines
- Modern React + Vite web application, interactive AI Tutor Copilot, and Flask REST API server

20. References

1. PyTorch Official Documentation

Comprehensive guide for recurrent neural layers (torch.nn.LSTM), tensor embeddings, and gradient optimization.

2. Deep Knowledge Tracing (NIPS 2015)

Piech, C., Bassen, J., Huang, J., Ganguli, S., Sahami, M., Guibas, L. J., & Sohl-Dickstein, J. *Deep Knowledge Tracing*. Advances in Neural Information Processing Systems (NeurIPS 2015).

3. Corbett & Anderson (1995) - Bayesian Knowledge Tracing

Corbett, A. T., & Anderson, J. R. *Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge*. User Modeling and User-Adapted Interaction, 4(4), 253-278.

4. Ebbinghaus, H. (1885) - Memory Decay Dynamics

Ebbinghaus, H. *Memory: A Contribution to Experimental Psychology*. Teachers College, Columbia University.

5. Vygotsky, L. S. (1978) - Zone of Proximal Development

Vygotsky, L. S. *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press.

6. ASSISTments 2009-2010 Benchmark Dataset

Feng, M., Heffernan, N. T., & Koedinger, K. R. *Addressing the Assessment Challenge with an Online System that Embodies Educational Theory*. Educational Data Mining benchmark repository.

7. Scikit-Learn Documentation

Reference standards for ROC-AUC calculations, accuracy scoring, confusion matrix analysis, and Brier calibration.

8. React & Vite Documentation

Modern frontend web framework and build tooling used for the reactive single-page student learning application.

9. Pandas & NumPy Documentation

High-performance tabular processing used for parsing Pipe-Separated Values (.psv) files and multi-dimensional tensor construction.

10. Open-Source Educational Data Mining & AIED Communities

International Educational Data Mining Society (IEDMS) and Artificial Intelligence in Education (AIED) open-source repositories and research forums.

THANK~YOU